DATABASE MANAGEMENT | CONCEPTS & DEMO

Short speaking guide based on the implemented local application

Kundan Pathak - 261010242 | Database Management
Project: NoteSphere - Smart Student Notes Sharing Platform

Your 45-60 second opening

"I am responsible for database management in NoteSphere. We use PostgreSQL, running locally through PGlite with data persisted on disk. The schema separates users, notes, file metadata, reviews, bookmarks and collections. Primary and foreign keys connect records, and unique constraints prevent duplicate bookmarks and multiple active reviews by one student on the same note. PDF bytes are stored outside the database; the files table stores their metadata. Indexed queries support search and filters. Numbered SQL migrations reproduce the schema, and private libraries are restricted by backend ownership checks."

What is the database actually using?

Local mode: @electric-sql/pglite embeds PostgreSQL in the application process and persists it at .data/postgres/. It is a relational SQL database, not localStorage, SQLite, MongoDB or MySQL. An external PostgreSQL option exists through the pg driver, but it is not needed for this local demo.

ENTITY / TABLES	WHAT THEY STORE OR CONNECT
users, sessions, password_resets	User/profile/role data and password hashes; hashed session/reset tokens and expiry times.
departments, courses, subjects	Academic reference data and course/department relationships.
notes, files	Note metadata, uploader, subject, visibility/status; storage key, MIME, size and file hash.
tags, note_tags	Reusable tags and a many-to-many note/tag relationship.
reviews, bookmarks	Student feedback/rating and saved note associations.
collections, collection_notes	Private named folders and a many-to-many collection/note relationship.
events, notifications, reports	Reading/download events, per-student updates and moderation reports.
admin_invitations, migrations	Hashed single-use invitation codes; schema migration history.

Relationship examples: One user has many notes; one note has many reviews. Many notes can belong to many collections through collection_notes. Notifications point to the recipient user and related note.

DATABASE MANAGEMENT | QUESTIONS & CODE MAP

Use these answers after rehearsing the actual code and demonstration

What makes the schema normalized?

Different concepts have separate tables. Reusable tags are not repeated as columns on every note; note_tags connects them. File metadata is separated from the note record. Keys preserve relationships and reduce inconsistent duplication.

Which constraints prevent invalid data?

Users have unique emails. reviews has UNIQUE(note_id, user_id), bookmarks has a combined primary key, and collection_notes also has a combined primary key. Rating checks allow 1-5; semester checks allow 1-12. Foreign keys reject broken relationships.

How does search remain efficient?

A GIN index supports PostgreSQL full-text search over note title, description and topic. Other indexes support visibility/status, subject/semester, owner and event lookups. The query uses websearch_to_tsquery and ranking; this is database search, not AI.

How are private collections protected?

The API scopes queries to the signed-in owner. SQL tables enable deny-by-default row-level security and revoke public access, but the trusted backend connects as schema owner. Therefore server-side authorization is still essential; do not claim per-user RLS alone enforces the app rules.

How do deletion and backup work?

Foreign keys use cascades for dependent rows, such as bookmarks and reviews when a note is deleted. File deletion is handled by the backend. Back up both .data/postgres/ and .data/files/ together while the local server is stopped; one PGlite directory must not be shared by multiple app processes.

Files and commands to show

db/001_initial.sql + db/002_role_portals.sql
Normalized schema, indexes, constraints, reference catalog and admin invitations.

db/demo-accounts.sql
Explicit local demo accounts. Stores password hashes; no fabricated notes or ratings.

src/lib/db.ts + src/lib/migrations.ts
Local/external connection selection and ordered SQL migration loading.

Command: npm run db:migrate
Demo: bookmark once, repeat the action, then sign out/in. Show that only one bookmark exists and it remains saved.